

PROJECT REPORT

AI Fitness & Diet Suggester

Bring Your Own Project (BYOP) — Capstone Submission

Name	Veer Jain
Reg No	25MIM10062
Branch	Integrated Mtech in AI
Year	2nd Year
Course Code	CSA2001
Course Title	Fundamentals in AI and ML
Language	Python 3
Libraries Used	pandas, scikit-learn, os, warnings
Interface	Command-Line (CLI)
Data Storage	CSV file (fitness_dataset.csv) + append-mode log CSV

1. Problem Statement

Me and two of my friends decided to start going to the gym around the same time. None of us really knew what we were doing so we kept asking our fourth friend for advice, he'd been going for a while and actually knew what he was talking about. The problem was all three of us had different goals and different builds. I was trying to lose some weight, one of them wanted to bulk up, and the other one just wanted to get more active in general. Our friend was giving each of us different advice which made sense, but he wasn't always free and we couldn't keep bothering him every time we had a question.

The obvious solution was to just look it up online but that honestly just made thing worse. Every website said something different.

The basic problem is that a decent fitness recommendation needs to actually account for who you are your age, your weight, what you're trying to achieve and not just throw a generic plan at everyone. I built this so the three of us could get a straight answer without having to bug our friend every other day.

2. How I Built It

2.1 Overall Structure

I kept the entire code in one Python script and split the logic into functions that collects inputs, trains the model, handles health snapshots (BMI , water intake, calories intake), prints the plan, and so on. The main() function just runs a loop with a numbered menu that calls whichever function the user picks.

I also included a small CSV dataset that the model trains on. If the file doesn't exist when you first run the program, it will be created automatically with 12 starter rows. That way it will work without any initial setup.

Component	What it does
<code>initialize_dataset()</code>	If the dataset doesn't exist it will create one
<code>train_model()</code>	It loads the CSV and fits a KNN classifier on age, weight, and goal
<code>get_user_profile()</code>	It collects and validates all inputs from the user (age, weight, height, gender, goal)
<code>show_bmi()</code> / <code>show_water_intake()</code> / <code>show_calorie_estimate()</code>	These 3 separate health snapshot calculations show before the main plan
<code>show_recommendation()</code>	It prints the workout, diet plan and a tip
<code>log_recommendation()</code>	It adds the session to a persistent CSV log
<code>view_logs()</code> / <code>clear_logs()</code>	It lets the user review or delete past recommendations

2.2 The Machine Learning Part

The actual recommendation comes from a K-Nearest Neighbours classifier from scikit-learn. I feed it three things about the user which are their age, weight in kg, and a goal code (0 for weight loss, 1 for muscle gain) and it predicts one of four workout types: Cardio, Weightlifting, Yoga, or HIIT.

I picked KNN because the logic it uses matches what I'd do manually. If someone is 22, weighs 85 kg, and wants to bulk up, they should get a similar or even maybe same recommendation to someone who is 24 and weighs 88 kg with the same goal. KNN literally finds the closest matching rows in dataset and goes with what those people got. I set $k=3$, mainly because the dataset is very small and going higher would have just averaged everything into mush.

2.3 Health Snapshot Calculations

Before the workout plan shows up, It displays: BMI, a daily water target, and a calorie intake estimate/recommendation. These don't involve the main model at all, they're just basic math on what the user entered. I added them because water and calorie intake are also a very big part of losing weight or having overall better health. I wanted people to also know their BMI and roughly how much to eat.

Calculation	Details
BMI	Standard formula: $\text{weight (kg)} / \text{height (m)}^2$. It is categorised as Underweight, Normal, Overweight, or Obese, with a short note for each.
Daily Water Intake	35ml per kg of body weight, converted to litres. Based on the most commonly cited figure across multiple sources.
Calorie Target (TDEE)	Mifflin-St Jeor BMR formula, it is adjusted for gender, multiplied by 1.55 (moderate activity). Deficit of 500 kcal for weight loss, surplus of 300 kcal for muscle gain.

2.4 Features

Feature	Notes
ML-based workout prediction	KNN model predicts Cardio, Weightlifting, Yoga, or HIIT based on user's input.
Full weekly workout plan	Each predicted workout type maps to a detailed day-by-day plan
Gender-specific workout tips	One targeted tip appended per plan based on gender
Goal-based diet plan	Two distinct nutrition plans: caloric deficit for weight loss, surplus for muscle gain
Health snapshot	BMI, daily water target, and TDEE-based calorie estimate before the main recommendation
Session log	Every recommendation is added to a CSV log with all user inputs and outputs
View and clear logs	User can review all past sessions or delete the log after confirmation

Validated input collection

Every input field loops until the user provides something valid which results in no crashes on bad input

3. Decisions I Made

KNN instead of if-else statement

My first version was just a chain of if-else statements. Weight loss? Recommend Cardio. Muscle gain? Recommend Weightlifting. It worked, but it completely ignored age and basically felt stupid and actually did nothing. A 50 year old and a 20 year old with the same goal are not going to get the same results from the same workout, and the program had no way to reflect that. KNN lets age and weight actually influence the output in a continuous way instead of forcing everyone into generic plan. It also means if I add more rows to the training CSV, the recommendations get better automatically which a static if else tree can't do.

Keeping height out of the model

I collect height from the user but I never pass it into the KNN model. The model predicts workout type, and honestly whether someone is 160 cm or 185 cm, it doesn't really change whether they should be doing Cardio vs. Weightlifting. Height actually only matters for BMI and calorie targets, so that's where I used it. Passing it into the mode would have just added noise.

Showing the health snapshot before the plan

I show the BMI, water intake, and calorie numbers before the workout recommendation, not after. I initially had it the other way around but it felt wrong when I was testing it. If someone's BMI puts them in the Overweight range, seeing that before the plan gives them some context for why they're getting HIIT. Without that, the recommendation could feel random from time to time.

Appending to the log instead of overwriting

I open the log file in append mode every time someone generates a recommendation. The whole point of keeping a log is to see what happened across multiple sessions if I overwrote it every run it would be useless. I also only write the header row if the file doesn't exist yet, so it doesn't end up with duplicate headers halfway through the file.

Requiring full 'yes' to clear the log

I made the clear confirmation check for the full word 'yes', not just the alphabet 'y'. I deleted everything with a quick 'y' a few times while testing and had to regenerate entries to check things. Requiring the whole word adds just enough friction too stop accidents like this. It's a small thing but it's the kind of detail I only added after I had already made the mistake.

4. Problems I Ran Into

scikit-learn printing warnings everywhere

The first few times I ran the program and my terminal was already full of DataConversion Warning messages from scikit learn. Nothing was actually broken, it was just the library being

annoying about internal stuff that the user doesn't need to know about. I added `warnings.filterwarnings('ignore')` at the top and moved on.

The calorie formula was wrong for half the users

The Mifflin-St Jeor BMR formula has different constants for males and females, +5 for male, -161 for female. In my original version I didn't ask for gender at all and just used the male constant for everyone. That means every female user was getting a calorie target that was over 150 kcal too high. I only noticed when I was double checking the formula against an online calculator and got different results. I added the gender input, fixed the formula, and the gender field also ended up being useful for the gender specific workout tips.

Height kept confusing the model

For a while I was passing height into the KNN model as a fourth feature. The problem was I only had 12 rows in the dataset. With four features instead of three, the nearest neighbours were being selected partly based on height similarity, which has nothing to do with what workout someone should be doing. The predictions got inconsistent, same age, weight, and goal but different heights would sometimes get completely different recommendations. I pulled height out of the model inputs and it immediately became more stable.

Nobody agrees on daily water intake

I looked up the recommended daily water intake across six different sources and got a different number from each of them. The range was roughly 30 ml to 40 ml per kg of body weight and so I went with 35 ml/kg because it appeared most often and it sits in the middle. The comment in the source code is pretty honest about this, I couldn't find one correct answer so I just went for the middle ground.

Bad inputs crashing the whole program

If you typed 'twenty' into the age prompt, `int('twenty')` would throw a `ValueError` and the whole thing would crash. I wrapped every input field in a `try/except` inside a while loop so it just asks again instead of dying. It should have been there from the start but I only added it after my friend typed their weight as '72.5 kg' instead of just '72.5' and the program exited on them.

5. What I Learned

- Fewer features can actually be better. I had four going into the model and the predictions were inconsistent. I dropped height and went down to three, and it got noticeably more stable. I had assumed more information always helps but it just doesn't when the dataset is small and the extra feature doesn't actually matter much.

- Simple looking formulas have hidden requirements. I missed that the calorie formula needed gender, BMI needs metric units, TDEE multipliers assume an activity level I had to pick in advance. None of which was obvious until I actually sat down and read what the formula was doing. I should have checked more carefully instead of assuming it will work everywhere.
- A tester is very important, since other people will use what I build in ways I had never even thought of. I would never have typed '72.5 kg' into a weight field but my roommate did it on day one. That's where the input validation came from. Since I know too much about the code and how it works that I never imagined for someone could also type that.
- Twelve rows is enough to show the idea works but not enough to trust the output in a real setting. The model works for the cases I put in the dataset but whether it holds up with a much wider range of users is a different question.

6. What Could Come Next

A few things I thought about but didn't build:

- A bigger dataset, 12 rows is enough to make the concept work but if this were real it would need a lot more variety before the predictions could be trusted across different body types and age groups.
- Fitness level as an input such as beginner, intermediate, advanced makes a meaningful difference, especially for HIIT intensity and where someone should start the weights with, but it was too complex and I didn't have much time for it in this version.
- Actually, doing something with the logs, they accumulate but nobody looks at them, even a basic summary like 'you've used this 5 times and always got Cardio' would make having logs worthwhile.
- Tkinter is on the list for next time, having a proper GUI would improve the looks as well as the overall feel of the project and dealing with CLI interface and making it look readable was super difficult.
- More workout categories, currently there are only four types covered which are the obvious cases but there's nothing for someone who's very underweight, has mobility issues, or is recovering from something.

7. Summary

Problem	People starting out with fitness have no easy way to get a personalised workout and diet recommendation without hiring a trainer or doing extensive research
----------------	--

Solution	CLI app that collects a short user profile, runs it through a trained KNN model, and outputs a full weekly workout plan, diet approach, and health snapshot
Language	Python 3
External Libraries	pandas (data loading), scikit-learn (KNN classifier); rest is standard library
ML Approach	K-Nearest Neighbours (k=3), trained on age, weight, and goal code
Data Storage	CSV for training data and recommendation log; no database needed at this scale
Concepts Applied	Supervised ML, feature engineering, BMR/TDEE formulas, input validation, file I/O, modular function design